
UNIT-IV

Arrays

Declaring and Referencing Arrays, Array Subscripts, Using Array Elements as Function Arguments, Array Arguments, Basic searching in an array of elements (linear and binary search techniques), Basic algorithms to sort array of elements (Bubble, Insertion and Selection sort algorithms two – dimensional arrays matrix addition and matrix multiplication, Declaration of Multidimensional arrays.

Arrays Concepts:

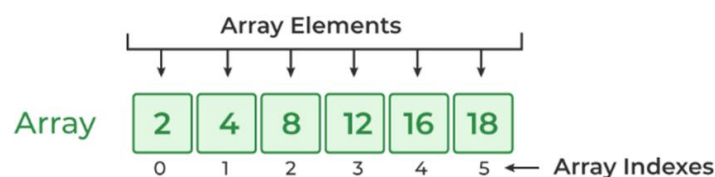
Structured programming is a programming technique in which a larger program is divided into smaller subprograms to make it easy to understand, easy to implement and makes the code reusable etc,. The structured programming enables code reusability. Code reusability is a method of writing code once and using it many times. Using structured programming technique, we write the code once and use it many times. Structured programming also makes the program easy to understand, improves the quality of the program, easy to implement and reduces time.

Array:

An array in C is a fixed-size collection of similar data items stored in contiguous memory locations.

An array is a collection of elements of the same type that are referenced by a common name.

It can be used to store the collection of primitive data types such as int, char, float, etc., and also derived and user-defined data types such as pointers, structures, etc.



Why Arrays:

Consider the following issue:

"We have a list of 1000 students' marks of an integer type. If using the basic data type (int), we will declare something like the following..."

```
int studMark0, studMark1, ...studMark999
```

Can you imagine how long we have to write the declaration part by using normal variable declaration?

```
int main(void)
```

```

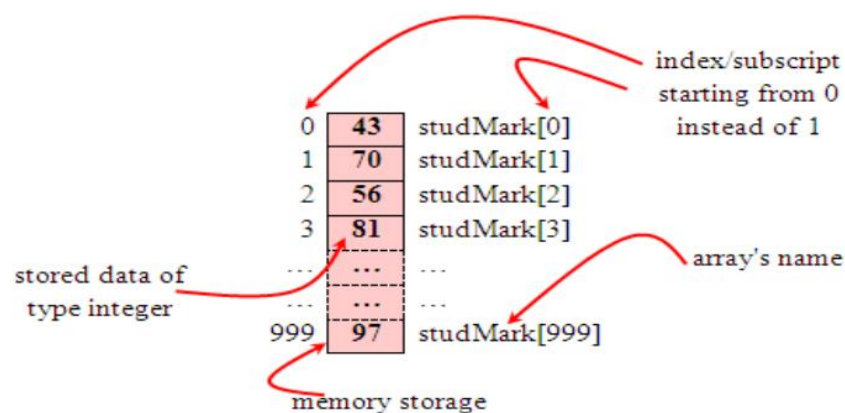
{
int studMark1, studMark2, studMark3,
studMark4, ..., ..., studMark998, stuMark999,
studMark1000;
...
...
return 0;
}

```

By using an array, we just declare like this,

```
int studMark[1000];
```

This will reserve 1000 contiguous memory locations for storing the students' marks. The complete set of students are represented using an array name students marks. A particular value is indicated by writing a number called index number or subscript in brackets after array name. The complete set of value is referred to as an array, the individual values are called elements. Graphically, this can be depicted as,.



Example of Arrays:

- List of employees in an organization.
- Test scores of a class of students.
- List of customers and their telephone numbers.
- List of students in the college.

For Example, to represent 100 students in college, can be written as

```
student [100]
```

Here student is a array name and [100] is called index or subscript.

We can use index or subscript to identify each element or location in the memory.

Hence, if we have an index of j, studMark[j] would refer to the jth element in the array of studMark.

For example, studMark[0] will refer to the first element of the array.

Thus by changing the value of j, we could refer to any element in the array. So, array has simplified our declaration and manipulation of the data.

CHARACTERISTICS OF AN ARRAY

An array represents a group of related data.

An array has two distinct characteristics:

An array is ordered: data is grouped sequentially: element 0, element n-1

An array is homogenous: every value within the array must share the same data type. In other words, an int array can only hold integers, but not other data types.

HOW TO CREATE AN ARRAY

Before we create an array, we need to decide on two properties:

Element type: what kind of data will the array hold? int, double, char? Remember, we can only choose one type.

Array size: how many elements will the array contain?

TYPES OF ARRAYS

Arrays can be classified based on how the data items will be arranged.

Arrays are broadly classified into three categories.

1. One dimensional arrays
2. Two dimensional arrays
3. Multi dimensional arrays

ONE DIMENSIONAL ARRAY:

One dimensional array is a linear list consisting of related and similar data items. In memory, all the data items are stored in contiguous memory locations.

Declaring a one dimensional array

To declare ordinary variables, we use the following notation:

```
int number;
```

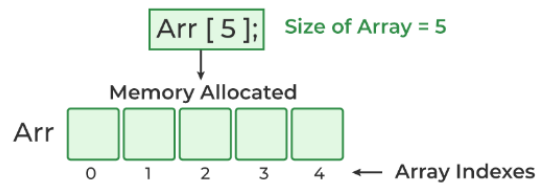
To declare an array, the following syntax can be used:

Syntax: array_element_data_type array_name [array_size];

For example:

```
int Arr[5];
```

Creates an array of 5 integer elements.



Here, `array_element_data_type` defines the base type of the array, which is the type of each element in the array.

`array_name` is any valid C identifier name that obeys the same rule for the identifier naming.

`array_size` defines how many elements the array will hold.

For example, to declare an array of 30 characters, that construct a people name, we could declare,

```
char cName[30];
```

In this statement, the array character can store up to 30 characters with the first character occupying location `cName[0]` and the last character occupying `cName[29]`.

Note that the index runs from 0 to 29. In C, an index always starts from 0 and ends with array's (size-1).

So, take note the difference between the array size and subscript/index terms.

Examples of the one-dimensional array declarations,

1. `int xNum[20], yNum[50];`
2. `float fPrice[10], fYield;`
3. `char chLetter[70];`

Array Initialization

Initialization in C is the process to assign some initial value to the variable.

When the array is declared or allocated memory, the elements of the array contain some garbage value. So, we need to initialize the array to some meaningful value.

multiple ways to initialize an array

1. Array Initialization with Declaration
2. Array Initialization with Declaration without Size
3. Array Initialization after Declaration (Using Loops)

1. Array Initialization with Declaration

Initialization of an array may take the following form

```
data_type array_name[size] = {a_list_of_value};
```

(or)

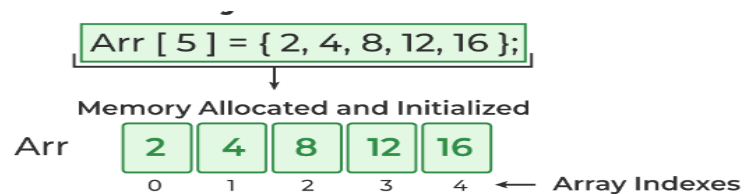
```
data_type array_name [size] = {value1, value2, ... valueN};
```

For example:

```
¢ int Arr[5] = {2, 4, 8, 12, 16};
```

```
¢ float fFloatNum[5] = {5.6, 5.7, 5.8, 5.9, 6.1};
```

```
¢ char chVowel[6] = {'a', 'e', 'i', 'o', 'u', '\0'};
```



Initialization of an array of type char for holding strings may take the following form,

```
char array_name[size] = "string_lateral_constant";
```

For example, char chVowel[6] = "aeiou";

For unsized array (variable sized), we can declare as follow,

```
char chName[ ] = "Mr. Johnson";
```

C compiler automatically creates an array which is big enough to hold all the initializer.

2. Array Initialization with Declaration without Size

The size of the array in these cases is equal to the number of elements present in the initializer list as the compiler can automatically deduce the size of the array.

```
data_type array_name[] = {1,2,3,4,5};
```

3. Array Initialization after Declaration (Using Loops)

initialize the array after the declaration by assigning the initial value to each element individually.

use for loop, while loop, or do-while loop to assign the value to each element of the array.

```
for (int i = 0; i < N; i++)
{
    array_name[i] = valuei;
}
```

Example of Array Initialization

```
#include <stdio.h>
```

```
int main() {
```

```
    // array initialization using initializer list
```

```

int arr[5] = { 10, 20, 30, 40, 50 };

// array initialization using initializer list without specifying size

int arr1[] = { 1, 2, 3, 4, 5 };

// array initialization using for loop

float arr2[5];

for (int i = 0; i < 5; i++) {
    arr2[i] = (float)i * 2.1;
}

return 0; }

```

Access/ Retrieving Array Elements

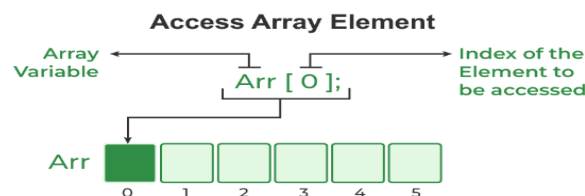
We can access any element of an array in C using the array subscript operator [] and the index value i of the element.

If you want to retrieve specific element then you have to specify not only the array or variable name but also the index number of interest.

Syntax

array_name [index];

One thing to note is that the indexing in the array always starts with 0, i.e., the first element is at index 0 and the last element is at $N - 1$ where N is the number of elements in the array.



Example - 1:

```

int Arr[]={1,3,5,6,8};

printf(“%d\t%d\n”,Arr[1],Arr[2]);

```

Output: 3 5

Example – 2:

```

#include <stdio.h>

int main() {
    int arr[5] = { 15, 25, 35, 45, 55 };
}

```

```

// accessing element at index 2 i.e 3rd element
printf("Element at arr[2]: %d\n", arr[2]);
printf("Element at arr[4]: %d\n", arr[4]);
printf("Element at arr[0]: %d", arr[0]);
return 0;
}

```

OUTPUT

Element at arr[2]: 35

Element at arr[4]: 55

Element at arr[0]: 15

Update Array Element

update the value of an element at the given index i in a similar way to accessing an element by using the array subscript operator `[ ]` and assignment operator `=`

Syntax:

```
array_name[i] = new_value;
```

Array Traversal

Traversal is the process in which, visit every element of the data structure.

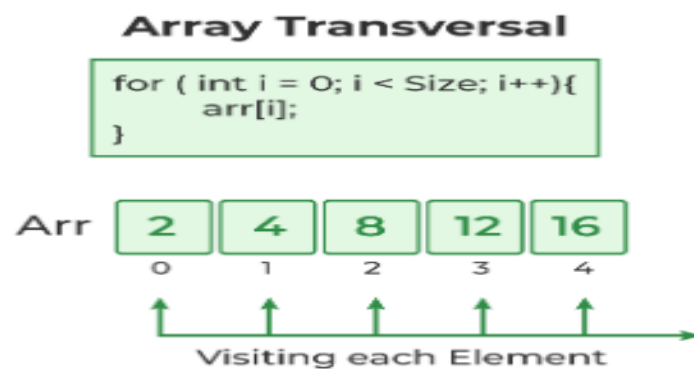
Array traversal, we use loops to iterate through each element of the array.

Syntax:

```

for (int i = 0; i < N; i++)
{
    array_name[i];
}

```



Example

```
#include <stdio.h>

int main()
{
    // array declaration and initialization
    int arr[5] = { 10, 20, 30, 40, 50 };
    // modifying element at index 2
    arr[2] = 100;
    // traversing array using for loop
    printf("Elements in Array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0; }
```

OUTPUT

Elements in Array: 10 20 100 40 50

Example

```
#include <stdio.h>

int main() {
    int arr[5];
    for (int i = 0; i < 5; i++) {
        arr[i] = i * i - 2 * i + 1;
    }
    printf("Elements of Array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0; }
```

OUTPUT

Elements of Array: 1 0 1 4 9

Example

```
#include <stdio.h>

int main()
{
    // creating array of character
    char arr[6] = { 'G', 'i', 'v', 'e', 'n', '\0' };

    // printing string
    int i = 0;
    while (arr[i]) {
        printf("%c", arr[i++]);
    }

    return 0;
}
```

OUTPUT

Given

ARRAYS AND FUNCTIONS

Passing an Array to a Function

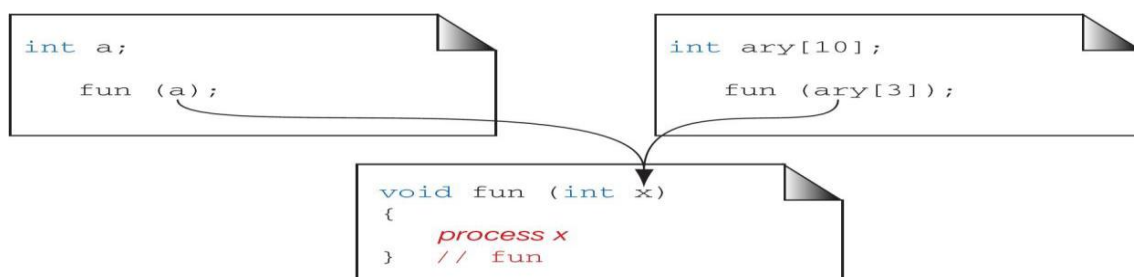
To process arrays in a large program, arrays are required to be passed to functions.

Arrays can be passed to functions in two ways:

1. Passing Individual Elements
2. Passing the whole Array

1. Pass Individual Array Elements

Passing array elements to a function is similar to passing variables to a function. Individual elements can be passed to a function either by passing the data values or by passing the addresses



Example

```
#include <stdio.h>

void display(int age1, int age2) {
    printf("%d\n", age1);
    printf("%d\n", age2);
}

int main() {
    int ageArray[] = {2, 8, 4, 12};
    // pass second and third elements to display()
    display(ageArray[1], ageArray[2]);
    return 0;
}
```

Output

8

4

2. Pass Arrays to Functions or Passing the whole Array

To pass an entire array to a function, only the name of the array is passed as an argument.

// Program to calculate the sum of array elements by passing to a function

```
#include <stdio.h>

float calculateSum(float num[]);

int main() {
    float result, num[] = {23.4, 55, 22.6, 3, 40.5, 18};
    // num array is passed to calculateSum()
    result = calculateSum(num);
    printf("Result = %.2f", result);
    return 0; }

float calculateSum(float num[]) {
    float sum = 0.0;
    for (int i = 0; i < 6; ++i) {
        sum += num[i]; }
}
```

```
    return sum;
}
```

Output

Result = 162.50

EXAMPLE PROGRAMS ON ONE DIMENSIONAL ARRAY:

1) Write a c program to find largest and smallest values in a list of array

```
#include<stdio.h>
#include<conio.h>

int main() {
    int ,i,n,x,y;
    printf("enter no.of elements ");
    scanf("%d",&n);
    int a[n];
    for(i=0;i<n;i++) {
        printf("\n enter %d value:");
        scanf("%d",&a[i]);
    }
    x=max(a,n);
    printf("the largest value is %d",x);
    y=small(a,n);
    printf("\nthe smallest value is %d",y);
}

int max(int arr[],int n)
{
    int largest,i; largest=arr[0]; for(i=0;i<n;i++)
    {
        if(arr[i]>largest)
        {
            largest=arr[i];
        }
    }
}
```

```

return(largest);
}
int small(int arr[],int n)
{
int smallest,i; smallest=arr[0]; for(i=0;i<n;i++)
{
if(arr[i]<smallest)
{
smallest=arr[i];
}
}
return(smallest);
}

```

Output

enter no.of elements 5

enter 0 value :10

enter 1 value :20

enter 2 value :30

enter 3 value :40

enter 4 value :50

the largest value is 50

the smallest value is10

// C Program to pass an array to a function as a address example

```

#include <stdio.h>

void printArray(int arr[])
{
    printf("Size of Array in Functions: %d\n", sizeof(arr));
    printf("Array Elements: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ",arr[i]);
    }
}

```

```

}

int main() {
    int arr[5] = { 10, 20, 30, 40, 50 };
    printf("Size of Array in main(): %d\n", sizeof(arr));
    printArray(arr);
    return 0;
}

```

Output

Size of Array in main(): 20

Size of Array in Functions: 8

Array Elements: 10 20 30 40 50

Return an Array from a Function

In C, we can only return a single value from a function. To return multiple values or elements, we have to use pointers. We can return an array from a function using a pointer to the first element of that array.

// C Program to return array from a function

```

#include <stdio.h>

int* func()
{
    static int arr[5] = { 1, 2, 3, 4, 5 };
    return arr;
}

int main() {
    int* ptr = func();
    printf("Array Elements: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", *ptr++);
    }
    return 0;
}

```

Output

Array Elements: 1 2 3 4 5

Linear Search

Linear search is a simple method where you check each element of the list one by one until the required value is found.

It Works on any type of list — sorted or unsorted.

Example

Suppose the array is:

[5, 12, 7, 9, 3]

To search for 9,

we check: 5 → No

12 → No

7 → No

9 → YES (Found)

Time Complexity:

Best case $O(1)$ (if found first)

Worst / Average case $O(n)$

Program:

```
#include<stdio.h>
#include<conio.h>
int main()
{
    int i,n,a[20],pos;
    clrscr();
    printf("Enter the number of elements:");
    scanf("%d",&n);
    printf("\nEnter the elements:");
    for(i=0;i<n;i++)
    {
        printf("enter element %d",i);
        scanf("%d",&a[i]);
    }
```

```

printf("enter the target element to be searched");
scanf("%d",&target);
pos=linearsearch(a[],n,target);
if(pos==-1)
printf("\n the target is not found in the list");
else
printf("\n the target %d is found at position
%d",target,pos);
return 0;
}
int linearsearch(int a[],int n, int target)
{
int i,j,temp;
for(i=0;i<n-1;i++)
{
if(a[i]==target)
return (i);
}
return -1;
}

```

Output:

Enter the number of elements:5

Enter the elements:1 3 6 5 4

Enter key

4

Key found at 3 position

Binary Search

Binary search is a faster search method that works only on sorted arrays.

It divides the list into halves and eliminates one half in each step.

Example:

Sorted Array: [2, 4, 6, 8, 10, 12, 14]

Search for 10:

Middle $\rightarrow$ 8 $\rightarrow$ not match

$10 > 8 \rightarrow$ search right half

Now array: [10, 12, 14]

Middle $\rightarrow$ 12 $\rightarrow$ not match

$10 < 12 \rightarrow$ search left

Now element = 10 $\rightarrow$ found

How It Works?

Find the middle element

If the middle value = key $\rightarrow$ found

If key < mid $\rightarrow$ search in left half

If key > mid $\rightarrow$ search in right half

Repeat until found or search space becomes empty

Time Complexity:

Best case $O(1)$

Worst / Average case $O(\log n)$

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[50], n, key, low, high, mid;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements in sorted order:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    printf("Enter the number to search: ");
```

```
    scanf("%d", &key);
```

```
    low = 0;
```

```

high = n - 1;
int found = 0;
while(low <= high) {
    mid = (low + high) / 2;
    if(arr[mid] == key) {
        found = 1;
        break;
    }
else if(key > arr[mid]) {
    low = mid + 1;
}
else {
    high = mid - 1;
}
}
if(found)
    printf("Element found at position %d", mid + 1);
else
    printf("Element not found");
return 0;
}

```

Duplicate Elements- Arrays

```

#include <stdio.h>

void findDuplicates(int arr[], int size) {
    printf("Duplicate elements in the sorted array: \n");
    for (int i = 0; i < size - 1; i++) {
        if (arr[i] == arr[i + 1]) {
            printf("%d ", arr[i]);

            // Skip the next element to avoid printing the same duplicate multiple times
            while (i < size - 1 && arr[i] == arr[i + 1]) {

```

```

        i++;
    }
}
}
printf("\n");
}

int main() {
    int arr[] = {1, 2, 2, 3, 3, 4, 5, 5, 5, 6};
    int size = sizeof(arr) / sizeof(arr[0]);
    findDuplicates(arr, size);
    return 0;
}

```

Duplicate Elements along with count-ARRAYS

```

#include <stdio.h>

void findDuplicatesWithCount(int arr[], int size) {
    printf("Duplicate elements with their counts in the sorted array: \n");
    for (int i = 0; i < size - 1; i++) {
        if (arr[i] == arr[i + 1]) {
            int count = 1; // Start count for the current duplicate element
            // Count the occurrences of the current duplicate element
            while (i < size - 1 && arr[i] == arr[i + 1]) {
                count++;
                i++; // Move to the next duplicate element
            }
            printf("Element %d appears %d times\n", arr[i], count);
        }
    }
}

int main() {
    int arr[] = {1, 2, 2, 3, 3, 4, 5, 5, 5, 6};

```

```
int size = sizeof(arr) / sizeof(arr[0]);  
findDuplicatesWithCount(arr, size);  
return 0; }
```

Bubble Sort

Bubble sort is one of the simplest sorting algorithms. It repeatedly compares adjacent elements and swaps them if they are in the wrong order. This process continues until the array becomes sorted.

Bubble Sort Working:

1. Start at the beginning of the array.
2. Compare each pair of adjacent elements.
3. Swap them if the left element is greater than the right element.
4. After one full pass, the largest element will “bubble up” to the end.
5. Repeat the passes, each time ignoring the last sorted elements.
6. Stop when no swaps are needed.

Bubble Sort Example

Let's take this array: [5, 1, 4, 2, 8]

Aim: sort it in ascending order using bubble sort.

<u>Pass</u>	<u>Result</u>
Pass 1	[1, 4, 2, 5, 8]
Pass 2	[1, 2, 4, 5, 8]
Pass 3	[1, 2, 4, 5, 8]
Pass 4	[1, 2, 4, 5, 8]

Program:

```
#include<stdio.h>  
  
int main()  
{  
int a[10],i,j,n,temp;  
printf("enter the size of an array");
```

```

scanf("%d",&n);

for(i=0;i<n;i++)
{
printf("enter the %d position element",i);
scanf("%d",&a[i]);
}
for(i=0;i<n-1;i++)
{
for(j=0;j<n-1-i;j++)
{
if(a[j]>a[j+1])
{
temp = a[j];
a[j] = a[j+1];
a[j+1] = temp;
}      }      }
printf("sorted elements are:");
for(i=0;i<n;i++)
{
printf("%d\t",a[i]);
}

return 0; }

```

```

enter the size of an array5
enter the 0 position element18
enter the 1 position element16
enter the 2 position element5
enter the 3 position element8
enter the 4 position element6
sorted elements are:5    6        8        16        18
-----
Process exited after 17.59 seconds with return value 0
Press any key to continue . . .

```

Optimized Bubble sort Program:

```
#include<stdio.h>

int main()
{
    int a[10],i,j,n,temp;
    printf("enter the size of an array");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("enter the %d position element",i);
        scanf("%d",&a[i]);
    }
    for(i=0;i<n-1;i++)
    {
        int swapped = 0;
        for(j=0;j<n-1-i;j++){
            if(a[j]>a[j+1])
            {
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
    printf("sorted elements are:");
    for(i=0;i<n;i++)
    {
        printf("%d\t",a[i]);
    }
    return 0; }
```

Selection Sort

- Partition the input list into a sorted and unsorted part (initially sorted part is empty)
- Select the smallest element and put it to the end of the sorted part
- Increase the size of the sorted part by one
- Repeat this n-1 times to sort a list of n elements

Selection Sort Example:

Sorted	Unsorted	
23	78 45 8 32 56	Original List
8 78	45 23 32 56	After pass 1
8 23 45	78 32 56	After pass 2
8 23 32 78	45 56	After pass 3
8 23 32 45 78	56	After pass 4
8 23 32 45 56	78	After pass 5

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        // Move minimum element to its correct position
        int temp = arr[i];
```

```

        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    } }

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n"); }

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    selectionSort(arr, n);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}

```

Time Complexity

Case	Time
Best (already sorted)	$O(n^2)$
Average	$O(n^2)$
Worst (reverse sorted)	$O(n^2)$

Insertion Sort

- Insertion Sort is a comparison-based sorting algorithm that builds the final sorted array one item at a time.
- the list is divided into two parts: sorted and unsorted.
- In each pass, the first element of the unsorted part is picked up, transferred to the sorted sub-list, and inserted at the appropriate place.

Ex: It works similarly to how you sort playing cards in your hand:

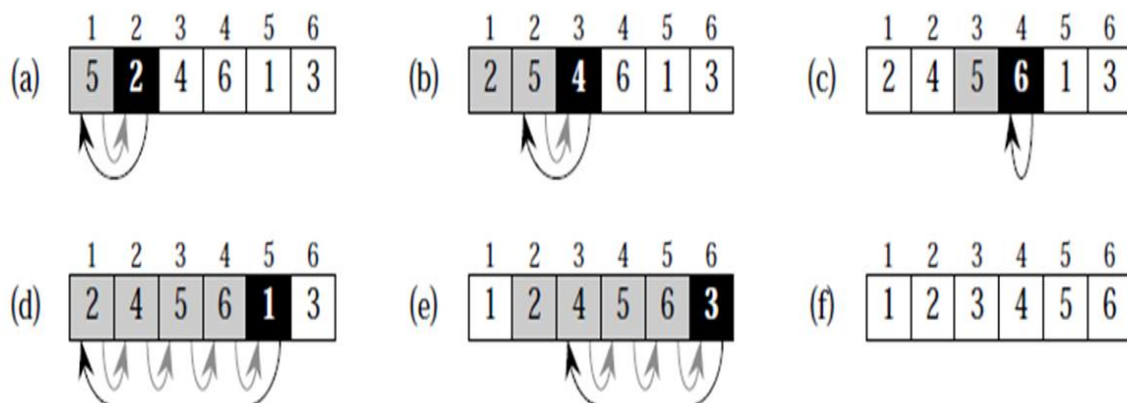
Insertion Sort Working:

1. You start from the second element.
2. Compare it with the elements before it.
3. Shift the larger elements to the right.
4. Insert the current element (the “key”) into its correct position.

Insertion Sort Example:

Let's take this array: [5, 2, 4, 6, 1, 3]

Aim: sort it in ascending order using insertion sort.



Example:

Sorted			Unsorted			
23	78	45	8	32	56	Original List
23	78	45	8	32	56	After pass 1
23	45	78	8	32	56	After pass 2
8	23	45	78	32	56	After pass 3
8	23	32	45	78	56	After pass 4
8	23	32	45	56	78	After pass 5

Insertion sort Program:

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
    printArray(arr, n);
    insertionSort(arr, n);
    printf("Sorted array: ");
    printArray(arr, n);
    return 0;
}
```

Time Complexity

Case	Time
Best (already sorted)	$O(n)$
Average	$O(n^2)$
Worst (reverse sorted)	$O(n^2)$

Comparison

Feature	Bubble Sort	Selection Sort	Insertion Sort
Best Case Time	$O(n)$	$O(n^2)$	$O(n)$
Average Time	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst Time	$O(n^2)$	$O(n^2)$	$O(n^2)$
Swaps	Many (slow)	Few (good)	Moderate
Stable	Yes	No	Yes
Adaptive? (Speeds up on nearly sorted data)	Yes	No	Yes
Overall Performance	✗ Slowest	✗ Not adaptive	☆ Best

MULTI – DIMENSIONAL ARRAYS

An array of arrays is called multi-dimensional array.

- A multidimensional array can have two dimensions, three dimensions, four dimensions, etc.
- A Multi-dimensional array which has only two dimensions is called as a two dimensional array.
- Two-dimensional array can be regarded as a table with rows and columns

Syntax

The general form of declaring N-dimensional arrays is shown as:

type arr_name[size1][size2][sizeN];

Examples:

Two-dimensional array: `int two_d [10][20];`

Three-dimensional array: `int three_d [10][20][30];`

Types of Multidimensional Arrays

many types of arrays depending on their dimensions but two of them are most commonly used:

Two-Dimensional Array (2D Array)

Three-Dimensional Array (3D Array)

Two-Dimensional (2D) Arrays

many types of arrays depending on their dimensions but two of them are most commonly used:

Two-Dimensional Array (2D Array)

Three-Dimensional Array (3D Array)

Declaration of 2D Array

```
type arr_name[m][n];
```

where,

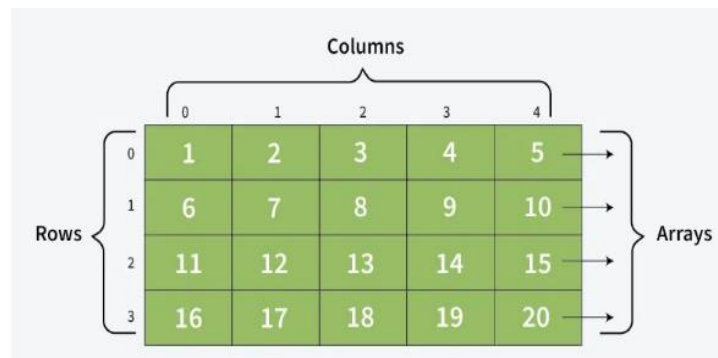
type: Type of data to be stored in each element.

arr_name: Name assigned to the array.

m: Number of rows.

n: Number of columns.

```
int arr[10][20];
```



Initialization of 2D Arrays

initialize a 2D array by using a list of values enclosed inside ‘{ }’ and separated by a comma.

Ex: `int arr[3][4] = {{0, 1, 2, 3}, {4, 5, 6, 7}, {8, 9, 10, 11}};`

or

```
int arr[3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};
```

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

Syntax:

```
type arr_name[][n] = {...values...};
```

Example:

```
char names [3][4] = { {'J', 'o', 'h', 'n'},
                      {'M', 'a', 'r', 'y'},
                      {'I', 'v', 'a', 'n'}
                    };
```

An integer array can be initialized to all zeros by using the following statement: `int nums[3][4] = {0};`

Access the Elements of a 2D Array

To access an element of a two-dimensional array, you must specify the index number of both the row and column.

Example:

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
printf("%d", matrix[0][2]); // Outputs 2
```

Change Elements in a 2D Array

To change the value of an element, refer to the index number of the element in each of the dimensions:

Example:

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
matrix[0][0] = 9;
printf("%d", matrix[0][0]); // Now outputs 9 instead of 1
```

Loop Through a 2D Array

Traversal means accessing all the elements of the array one by one

An element in two-dimensional array is accessed using row indexes and column indexes inside array subscript operator `[]`.

To traverse the whole array, we will use two loops. One loop to go over each row from top to bottom and the other nested inside it to access each element in the current row from left to right.

Example:

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };  
int i, j;  
for (i = 0; i < 2; i++) {  
    for (j = 0; j < 3; j++) {  
        printf("%d\n", matrix[i][j]);  
    }  
}
```

OUTPUT

```
1  
4  
2  
3  
6  
8
```

row-by-row traversal of a 2D array

```
#include <stdio.h>  
  
int main() {  
    // Create and initialize an array with 3 rows and 2 columns  
    int arr[3][2] = { { 0, 1 }, { 2, 3 }, { 4, 5 } };  
    // Print each array element's value  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 2; j++) {  
            printf("arr[%d][%d]: %d  ", i, j, arr[i][j]);  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

Output

arr[0][0]: 0 arr[0][1]: 1

arr[1][0]: 2 arr[1][1]: 3

arr[2][0]: 4 arr[2][1]: 5

Addition of 2 matrices:

$$A = \begin{bmatrix} 3 & 4 & 5 \\ 6 & 7 & 8 \\ -10 & -11 & -12 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} -1 & -2 & -3 \\ 4 & 12 & 1 \\ 19 & 9 & 3 \end{bmatrix}$$

$$A+B = \begin{bmatrix} 3-1 & 4-2 & 5-3 \\ 6+4 & 7+12 & 8+1 \\ -10+19 & -11+9 & -12+3 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \\ 10 & 19 & 9 \\ 9 & -2 & -9 \end{bmatrix}$$

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int a[100][100],b[100][100],c[100][100];
```

```
int i,j,n,m;
```

```
printf("enter row size of the matrix(<100)"); scanf("%d",&n);
```

```
printf("enter column size of the matrix(<100)"); scanf("%d",&m);
```

```
printf("enter the matrix a");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
For(j=0;j<m;j++)
```

```
{
```

```
scanf("%d",&a[i][j]);
```

```
} }
```

```
Printf("enter the matrix b"); For(i=0;i<n;i++)
```

```
{
```

```
For(j=0;j<m;j++)
```

```
{
```

```
Scanf("%d",&b[i][j]);
```

```
} }
```

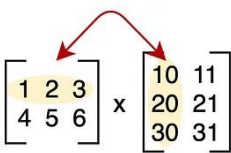
```
printf("MATRIX a+MATRIX b");
```

```

For(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
c[i][j]=a[i][j]+b[i][j];
}
}
printf("addition of two matrices is\n"); For(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
Printf("%d\t",c[i][j]);
}
}
return 0;
}

```

Multiplication of two matrices:



$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

Matrix Multiplication Rule

You can multiply two matrices A ($m \times n$) and B ($n \times p$) only if:

Number of columns of A = Number of rows of B

Example:

A is 2×3 and B is $3 \times 4 \rightarrow$ multiplication possible $\rightarrow$ result will be 2×4 .

If they do NOT satisfy the rule $\rightarrow$ multiplication not possible.

```

#include <stdio.h>

int main() {
    int a[10][10], b[10][10], c[10][10];
    int r1, c1, r2, c2;

    // Input sizes
    scanf("%d %d", &r1, &c1); // rows and columns of A
    scanf("%d %d", &r2, &c2); // rows and columns of B

    // Rule check
    if (c1 != r2) {
        printf("Matrix multiplication not possible");
        return 0;
    }

    // Input matrix A
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c1; j++)
            scanf("%d", &a[i][j]);

    // Input matrix B
    for (int i = 0; i < r2; i++)
        for (int j = 0; j < c2; j++)
            scanf("%d", &b[i][j]);

    // Initialize result matrix C with 0
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++)
            c[i][j] = 0;

    // Matrix multiplication
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            for (int k = 0; k < c1; k++) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

```

```
    }  
}  
// Print result  
for (int i = 0; i < r1; i++) {  
    for (int j = 0; j < c2; j++) {  
        printf("%d ", c[i][j]);  
    }  
    printf("\n");  
}  
return 0;  
}
```